

V4L2

Author: Brian Kang
Create: 4/16/2026
Last Modified: 8/12/2026

Preface

Overview

This Application Note provides a comprehensive technical overview of the **Video for Linux 2 (V4L2)** framework, the standard API for video capture and output on Linux-based systems. As embedded systems and multimedia applications demand higher performance and lower latency, understanding the architectural nuances of V4L2 becomes essential for developing robust imaging and video solutions.

Purpose of this Document

The primary objective of this document is to guide developers through the core concepts, data structures, and operational flows of the V4L2 framework. It focuses on practical implementation strategies, including device configuration, buffer management, and efficient I/O handling. By detailing the interaction between userspace applications and kernel-side drivers, this guide aims to simplify the complexity of managing video streams in an embedded Linux environment.

1 Pixel Format

Video is a moving Picture. It means, continuously draw pictures 25 to 60 times per second.
Human eye recognize as video if picture repeated more than 24 times for sec.
The picture is consisted with pixel that can be RGB or YUV on components.



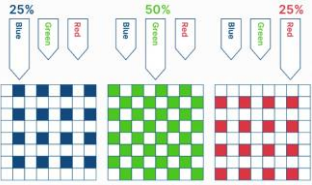
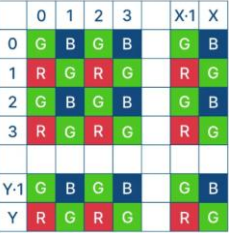
1.1 Color (RGB and YUV)

Primary color is Red, Green, and Blue. If we combine together, we can make other colors. For example, if Red is 128, Green is 128 and Blue is 0, it becomes Yellow color. If all RGB has same value, it generates gray color. The big value becomes bright, the small value becomes blank.

The Alpha is a forth component, the 0xFF value means solid, and 0x00 means totally un-visible.

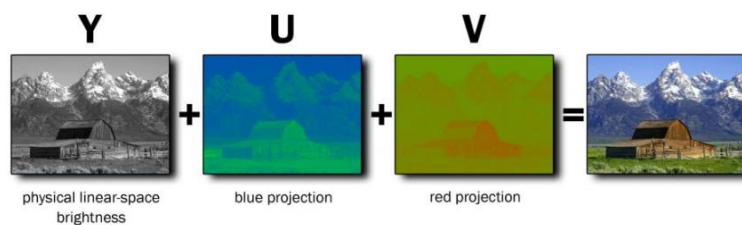
V4L2_PIX_FMT_	code	bits	Pattern (Packed)
RGB24	RGB3	24	R G B, R G B, ...
BGR24	BGR3	24	B G R, B G R, ...
ABGR32	AR24	32	B G R A, B G R A, ...
XBGR32	XR24	32	B G R x, B G R x, ... (x means do not care about the value)
ARGB32	BA24	32	A R G B, A R G B, ... (deprecated)
XRGB32	BX24	32	x R G B, x R G B, ... (deprecated)

The camera CCD sensor, it generates the Bayer RGB format. Only a color filter on the sensor makes each pixel sensitive to a specific color - red, green, or blue. And then camera ISP(Image Signal Processor), it converts Bayer GRB to the popular RGB or YUV format.

V4L2_PIX_FMT_	code		Bayer Patterns	Example of GBRG
SBGGR8	BA81		B G B G G R G R ...	
SGBRG8	GBRG		G B G B R G R G ...	
SGRBG8	GREG		G R G R B G B G ...	
SRGGB8	RGGB		R G R G G B G B ...	

YUV color is composed with separated with Luminance and Color component (Chrominance).

If we use 8Bit for Y, 8Bit for U, and 8Bit for U, we call it as YUV444, and it can be mathematically convert from RGB or to RGB without loss.



Human eyes are sensitive on Luminance, and less sensitive on color. We can use two Y and one UV to reprehensive two pixels, In this case it uses 16bit that was 24bit on RGB. So, it can reduce 1/3 of data size. We call it as YUV422. On the video codec, that want to reduce some more. It uses 4 Ys, and one color component to represent 4 pixels. It uses 12bit and we call is as YUV420.

On YUV, we can separate the color component on the separate group.

2 Plane mode, using 1 group for Y, and use 1 group for UV.

3 Plane mode, using 1 group for Y, 1 group for U, and 1 group V.

So, UYVY is packed, and NV16, NV12 are planned.

YUV formats. Most frequently used format is UYVY or YUYV.

V4L2_PIX_FMT_	code	YUV	Plane	
YUYV	YUYV	422	1	Y0 Cb Y1 Cr, ...
UYVY	UYVY	422	1	Cb Y0 Cr Y1, ...
NV24	NV24	444	2	Plane0 Y Y Y Y, ..., Plane1 Cb Cr Cb Cr, ...
NV16	NV16	422	2	Plane0 Y Y Y Y, ..., Plane1 Cb Cr Cb Cr, ...
NV12	NV12	420	2	Plane0 Y Y Y Y, ..., Plane1 Cb Cr, ...
YUV444M	YM24	444	3	Plane0 Y, ..., Plane1 Cb, Plane2: Cr, ...
YUV422P	422P	422	3	Plane0 Y Y, ..., Plane1 Cb, ..., Plane2: Cr, ...
YUV420	YV12	420	3	Plane0 Y Y Y Y, ..., Plane1 Cb, ..., Plane2: Cr, ...
GREY	GREY			Y onlyl.
YUV32	YUV4			32bit. A Y Cb Cr, packed.

The Video Codec normally using Planed format (NV16, NV12, 422P, or YV12).

1.2 Color Conversion

RGB to YUV or YUV to RGB

Standard: BT601, BT709, BT2020 (BT656 will be explain later)

Consider Full and Extension

V4L2 identifier	Nick Name	
V4L2_COLORSPACE_SMPTE170M	SD	CVBS(NTSC, PAL), VGA(640x480),
V4L2_COLORSPACE_REC709	HD	720P, 1080P
V4L2_COLORSPACE_BT2020	UHD	4K vide

Quantization Methods

Normally RGB values are full range. If we use each 8bytes, each R,G,B are mapped to [0..255].

YUV can be full range, but we have to consider the limited range Y[16..235], UV[16..240] that was used on old SD(Standard Definition) format.

Identifier	
V4L2_QUANTIZATION_DEFAULT	This is always full range for R'G'B' (except for the BT.2020 colorspace) and usually limited range for Y'CbCr.
V4L2_QUANTIZATION_FULL_RANGE	[0..255]
V4L2_QUANTIZATION_LIM_RANGE	Y is mapped to [16..235], CbCr is mapped to [16..240]

2. Video Format

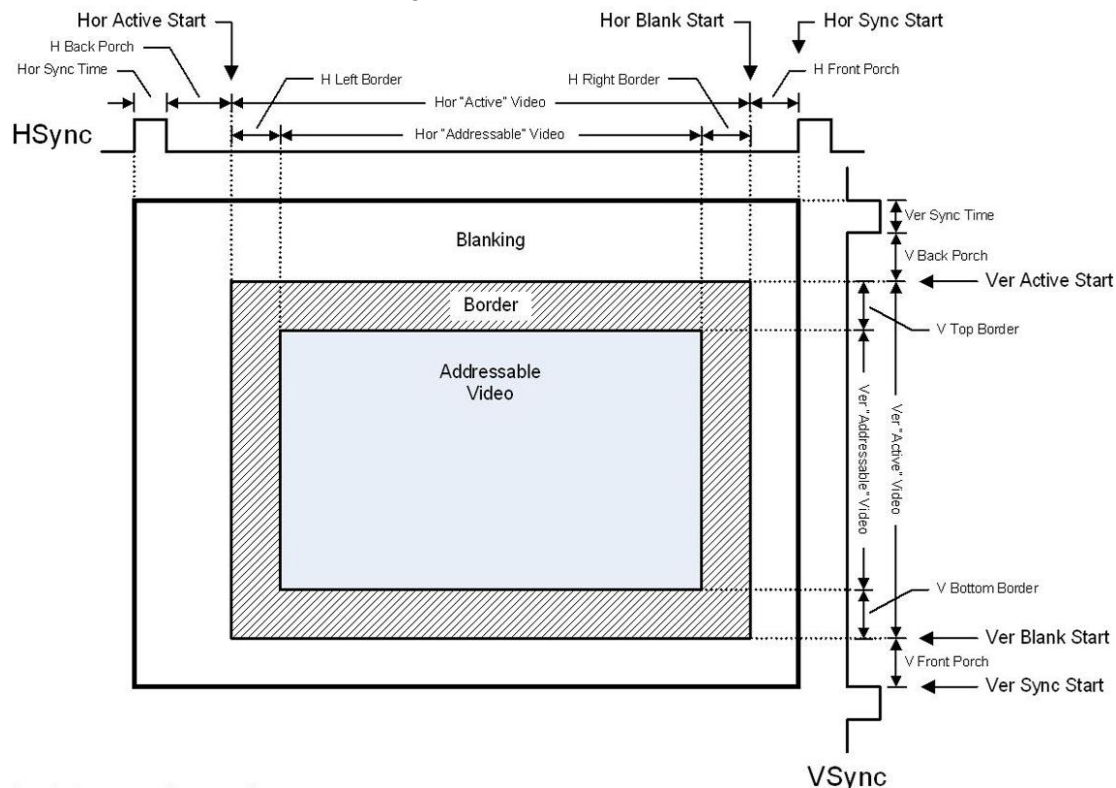
2.1 Resolution

Resolution means a Video size.

H x V	Name	Description
640x480	VGA	
720x480	SD	
800x480	WGA	Wide VGA
800x600	SVGA	Super VGA
1024x768	XGA	
1280x720	XWGA	HD
1920x1080	FHD	
2560x1440	QHD	2K
3840x2160	UHD	4K

2.2 Video Control Signals

Video Control Signals provide the spatial and temporal "map" for the raw pixel data. It provides where one line ends or where a new frame begins.



item	Description
hActive	Horizontal valid pixels
hBlank	blank area on Horizontal Line. hFrontPorch + hSync + hBackPorch
hFrontPorch	space between last valid pixel and hSync

hSync	Horizontal Sync Signal area.
hBackPorch	space between hSync and the first valid pixel
hTotal	hActive + hBlank
vActive	Vertical valid pixel lines
vBlank	blank area on Vertical Lines. vFrontPorch + vSync + vBackPorch
vFrontPorch	space between last valid line and vSync
vSync	Vertical Sync Signal area
vBackPorch	space between vSync and first valid line
vFreq	How many video frame need to display on 1 sec. ex:60Hz,59.49,50,..
Pixel Rate	Time to draw one pixel. hTotal * vTotal * vFreq. ex:720P60 needs 74.25MHz
DE	Data Enabled, same as hActive.

3 V4L2 capture

Below is a simplest capture program steps. It will be explained step-by-step.

```

vfd = open("/dev/video0");
ioctl(vfd, VIDIOC_QUERYCAP, &cap);

ioctl(vfd, VIDIOC_S_FMT, &fmt);
ioctl(vfd, VIDIOC_REQBUFS, &reqbuf); /* with NUM_OF_BUFFERS count */
for(i = 0; i < NUM_OF_BUFFERS; i++) {
    ioctl(vfd, VIDIOC_QUERYBUF, &buffer);
    vin_pool[i].plane[0] = mmap(...)
    ret = ioctl(vfd, VIDIOC_QBUF, &buf);
}

ioctl(vfd, VIDIOC_STREAMON, &type);
while(1) {
    ioctl(vfd, VIDIOC_DQBUF, &buf);
    printf("%d ", buf.index);
    ioctl(vfd, VIDIOC_QBUF, &buf);
}
ioctl(vfd, VIDIOC_STREAMOFF, &type);
for(i = 0; i < NUM_OF_BUFFERS; i++) {
    munmap(..);
}
ioctl(vfd, VIDIOC_REQBUFS, &reqbuf); /* with 0 count */
close(vfd);

```

3.1 Open V4L2 driver

camera capture driver is generated on "/dev/videoX" (X means number). At default open and save at vin_fd variable.

```
vin_fd = open("/dev/video0", O_RDWR);
```

Next step is check VIDEO_CAPTURE and STREAMING feature.

```

struct v4l2_capability cap;
u32 caps;
ret=ioctl(vfd, VIDIOC_QUERYCAP, &cap);

```

```

#if LINUX_VERSION_CODE >= KERNEL_VERSION(3,10,0)
    caps = cap.device_caps;
#else
    caps = cap.capabilities;
#endif
assert(caps & V4L2_CAP_VIDEO_CAPTURE);
assert(caps & V4L2_CAP_STREAMING);

```

As you can see, we need `KERNEL_VERSION()` because V4L2 is updating.

3.2 Data Format

`VIDIOC_S_FMT` and `VIDIOC_G_FMT` with `v4l2_format`.

<code>VIDIOC_S_FMT</code>	Its locking side effects are implied by the next step, the selection of an I/O method with the ioctl <code>VIDIOC_REQBUFS</code> ioctl
<code>VIDIOC_G_FMT</code>	It returns the format that was assigned by <code>S_FMT</code> , or internal default value.
<code>VIDIOC_TRY_FMT</code>	to examine what the hardware <i>could</i> do, without actually selecting a new data format.
<code>VIDIOC_ENUM_FMT</code>	to enumerate all image formats supported by video capture, overlay or output devices

```

struct v4l2_format {
    __u32 type;
    union {
        struct v4l2_pix_format;
        ...
    } fmt;
};

```

```

struct v4l2_pix_format {
    __u32 width, __u32 height;
    __u32 pixelformat;
    enum v4l2_field field;
    __u32 bytesperline;
    __u32 sizeimage;
    enum v4l2_colospace;
    __u32 priv;
    __u32 flags;
    enum v4l2_ycbcr_encoding;
    enum v4l2_quantization;
    enum v4l2_xfer_func;
};

```

Item	Description
height	Normally, number of lines on frame. On <code>V4L2_FIELD_ALTERNATE</code> (or <code>TOP</code> , <code>BOTTOM</code>), number of lines in field. So, NTSC video with <code>V4L2_FIELD_INTERLACED</code> (or <code>_TB</code>)uses 480.
<code>v4l2_field</code>	Field Order. The progressive uses <code>V4L2_FIELD_NONE</code> . NTSC uses <code>V4L2_FIELD_INTERLACED</code> or <code>V4L2_FIELD_INTERLACED_TB</code> .
bytesperline	On RGB, it is width * 3. On UYVY, it is width * 2.
<code>v4l2_colospace</code>	<code>V4L2_COLORSPACE_SMPTE170M</code> , <code>V4L2_COLORSPACE_REC709</code> , or

	V4L2_COLORSPACE_BT2020
v4l2_quantization	V4L2_QUANTIZATION_FULL_RANGE, V4L2_QUANTIZATION_LIM_RANGE, or V4L2_QUANTIZATION_DEFAULT

1280x720 UYVY format

```
struct v4l2_format fmt = {0};

fmt.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
fmt.fmt.pix.width      = 1280;
fmt.fmt.pix.height     = 720;
fmt.fmt.pix.field      = V4L2_FIELD_NONE;
fmt.fmt.pix.pixelformat = V4L2_PIX_FMT_UYVY;
ret=ioctl(vfd, VIDIOC_S_FMT, &fmt);
```

Why it needs VIDIOC_S_FMT? Sometimes the default value on device driver is incorrect, and the next VIDIOC_REQBUF uses the assigned value by this VIDIOC_S_FMT. You can use VIDIOC_G_FMT to read the current setting value.

3.3 Streaming I/O

V4L2 supports read(), and write(), but, in most of case, it using streaming I/O method for the big data size.

IO method	Description
Memory Mapping	Memory mapping is primarily intended to map buffers in device memory into the application's address space.
User Pointers	Buffers (planes) are allocated by the application itself. If required by the hardware, the driver swaps memory pages within physical memory to create a continuous area of memory.
DMA buffer	The DMABUF framework provides a generic method for sharing buffers between multiple devices.

The recommend method for camera VIN is using V4L2_MEMORY_MMAP, and do export DMA if you need DMA processing.

```
#define NUM_OF_BUFFERS 4

struct v4l2_requestbuffers reqbuf = {0};
reqbuf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
reqbuf.memory = V4L2_MEMORY_MMAP;
reqbuf.count = NUM_OF_BUFFERS;
ret = ioctl(vin_fd, VIDIOC_REQBUFS, &reqbuf);

for(i = 0; i < NUM_OF_BUFFERS; i++) {
    struct v4l2_buffer buffer;
    memset(&buffer, 0, sizeof(*buffer));
    buffer.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
    buffer.memory = V4L2_MEMORY_MMAP;
    buffer.index = i;
    ret = ioctl(vin_fd, VIDIOC_QUERYBUF, &buffer);

    vin_pool[i].image_size = buffer.length;
    vin_pool[i].plane[0] = mmap( NULL, buffer.length,
        PROT_READ | PROT_WRITE,
        vin_fd, buffer.m.offset);
}
#endif USE_DMABUF
```

```

for(i = 0; i < NUM_OF_BUFFERS; i++) {
    struct v4l2_exportbuffer expbuf = {0};
    expbuf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
    expbuf.index = i;
    ret = ioctl(vdev->fd, VIDIOC_EXPBUF, &expbuf);
    vin_pool[i].dmafd[0] = expbuf.fd;
}
#endif

```

On R-CAR H3, it using 3 HW slot, so minimum required frames are 4.

3.4 Receive the captured frame

QUEUE	<pre> /* send buffer to driver's READY_QUEUE */ int vin_qbuf(int vin_fd, int buf_idx) { struct v4l2_buffer buf = {0}; buf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE; buf.memory = V4L2_MEMORY_MMAP; buf.index = buf_idx; ret = ioctl(vin_fd, VIDIOC_QBUF, &buf); } </pre>
DQUEUE	<pre> /* get the captured buffer from drivers DONE_QUEUE */ int vin_dqbuf(int vin_fd, int *buf_idx, u32 *seq, struct timeval *ts) { struct v4l2_buffer buf = {0}; buf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE; buf.memory = V4L2_MEMORY_MMAP; ret = ioctl(vin_fd, VIDIOC_DQBUF, &buf); if(buf.flags & V4L2_BUF_FLAG_ERROR) { printf("ERROR"); return 1; } *buf_idx = buf.index; if(seq) *seq = buf.sequence; if(ts) { ts->tv_sec = buf.timestamp.tv_sec; ts->tv_usec = buf.timestamp.tv_usec; } return 0; } </pre>
STREAM ON	<pre> int vin_stream_on(int vin_fd) { int ret; int type = V4L2_BUF_TYPE_VIDEO_CAPTURE; ret=ioctl(vin_fd, VIDIOC_STREAMON, &type); return ret; } </pre>
STREAM OFF	<pre> int vin_stream_off(int vin_fd) { int ret; int type = V4L2_BUF_TYPE_VIDEO_CAPTURE; </pre>


```

        ret=ioctl(vin_fd, VIDIOC_STREAMOFF, &type);
        return ret;
    }

```

At first, we send all available buffers to driver. After streamon, we can receive the captured frame. As soon as we received, we will send it back to driver, because R-CAR H3 needs minimum 3 buffers on the its ready queue.

```

int buf_idx;
int i;
for(i=0; i < NUM_OF_BUFFERS; i++)
    vin_qbuff(vin_fd, i);

vin_stream_on(vin_fd);
while(1) {
    if(kbhit())
        break;
    vin_dequeue(vin_fd, &buf_idx, NULL, NULL);
    printf("%d ", buf_idx);
    vin_qbuf(vin_fd, buf_idx);
}
vin_stream_off(vin_fd);

```

3.5 BLOCKED IO and epoll()

When we opened driver, we did not use "O_NONBLOCK". So when we execute VIDIOC_DQBUF, it will wait until the driver has a captured frame on DONE_QUEUE. Even we add kbhit() function, but program will be blocked within ioctl(vin_fd, VIDIOC_DQBUF, &buf) on the no signal situation.

```

vin_fd = open("/dev/video0", O_RDWR);

```

To solve, use select() or epoll() that is activated when it has a captured frame.

```

static int vin_register_poll(int vin_efd, int flag)
{
    struct epoll_event event = {0};
    int ret;

    event.events = EPOLLIN;
    event.data.u32 = 0; /* for cam_idx, 0~3 */

    ret = epoll_ctl(
        vin_efd,
        (flag ? EPOLL_CTL_ADD : EPOLL_CTL_DEL),
        vin_fd, &event);
    CHK_RET(ret);

    TRACE(INFO, _b("#%d: poll source %s"), cam_idx, (flag ? "added" : "removed"));

    return 0;
}

```

```

#include <sys/poll.h>
#include <sys/epoll.h>

#define MAX_VIN_DEVS 1
struct epoll_event event[MAX_VIN_DEVS];
int vin_efd = 0;

int buf_idx;
int i;
int timeout = 40; /* 40ms */
for(i=0; i < NUM_OF_BUFFERS; i++)
    vin_qbuff(vin_fd, i);

vin_efd = epoll_create(MAX_VIN_DEVS);

vin_register_poll(vin_efd, 1); /* EPOLL_CTL_ADD */
vin_stream_on(vin_fd);
while(1) {
    int r, k;

    if(kbhit())
        break;
    r = epoll_wait(vin_efd, MAX_VIN_DEVS, timeout);
    if(r < 0) {
        printf("Error\n");
        break;
    }
    else if(r == 0) {
        /* timeout
        continue;
    }

    for(k=0; k < r; k++) {
        assert(event[k].data.u32 == 0); /* for video0 */
        if((event[k].events & EPOLLIN) == 0) {
            if(((event[k].events & EPOLLERR) == 0)
                continue;
            printf("Invalid poll events = 0x%X\n", event[k].events);
            continue;
        }
        vin_dequeue(vin_fd, &buf_idx, NULL, NULL);
        printf("%d ", buf_idx);
        vin_qbuf(vin_fd, buf_idx);
    }
}
vin_register_poll(vin_efd, 0); /* EPOLL_CTL_DEL */
vin_stream_off(vin_fd);

```

Upper code executes VIDIOC_DQBUF only when it has EPOLLIN event.

3.5.1 poll example

poll() is the simpler mechanism and older than epoll(). Every time you call it, you pass an array of pollfd structures from userspace to the kernel.

```
#include <sys/poll.h>

drm_create("rcar-du", &g_drm_pix, &drm_fd);
vin_capture_create(&g_vin_pix, &vin_fd);

struct pollfd fds[] = {
    { .fd = vin_fd, .events = POLLIN },
    { .fd = drm_fd, .events = POLLIN }, /* For future DRM_MODE_PAGE_FLIP_EVENT */
};

ret=vin_capture_start();

while(1) {
    ret = poll(fds, 2, 500); /* 0.5s */
    if(ret < 0)
        break;
    if(ret == 0 && kbhit()) /* type any key to quit */
        break;

    //ret can be 1 or 2.
    //if(fds[0].revents & POLLIN) { handle new camera frame };
    //if(fds[1].revents & POLLIN) { handle page flip event. need drmHandleEvent() };

    buf_idx = vin_capture_frame();
    drm_draw(drm_fd, buf_index);
    vin_dispose_frame(buf_index, 0);
}
vin_capture_stop();
drm_destroy();
printf("ByeBye\n");
```

ref: <https://www.kernel.org/doc/html/v4.9/media/uapi/v4l/func-poll.html#func-poll>

3.5.2 select example

select() requests the highest fd number that you are using.

```
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>
#define CLEAR(x) memset(&(x), 0, sizeof(x))

while(1) {
    fd_set fds;
    struct timeval tv;
    int r;

    FD_ZERO(&fds);
    FD_SET(fd, &fds);
```

```

/* Timeout. */
tv.tv_sec = 2;
tv.tv_usec = 0;

r = select(fd + 1, &fds, NULL, NULL, &tv); /* highest fd + 1 */
if (-1 == r) {
    if (EINTR == errno)
        continue;
    errno_exit("select");
}
if (0 == r) {
    fprintf(stderr, "select timeout\\n");
    exit(EXIT_FAILURE);
}
if (read_frame())
    break;
}

```

3.6 Release buffer and close V4L2 driver

When you finish, you have to unmap the memory, and release the allocated frames.

```

for(i = 0; i < NUM_OF_BUFFERS; i++) {
    munmap(buffers[i].plane[0], vin_pool[i].length);
}

#ifdef USE_DMABUF
for(i = 0; i < NUM_OF_BUFFERS; i++)
    close(vin_pool[i].dmafd[0]);
#endif

struct v4l2_requestbuffers reqbuf = {0};
reqbuf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
reqbuf.memory = V4L2_MEMORY_MMAP;
reqbuf.count = 0; /* 0 for cleanup */
ret = ioctl(vin_fd, VIDIOC_REQBUFS, &reqbuf);

close(vin_fd);

```

3.7 VIN Device Driver

V4L2 framework is on drivers/media/v4l2-core and R-CAR implementation is on drivers/media/platform/rcar-vin, and my camera interface drivers are on drivers/media/i2c. And DTS files are on arch/arm64/boot/dts.

```

/
+- arch/arm64/boot/dts/
| +- r8a779m1-ulcb-9974.dts
| +- r8a779m1-ulcb-CeX-8842.dts
| +- r8a779m1-ulcb-CeX-8846.dts
| +- r8a779m1-ulcb-CeX-9972.dts

```

```

| +- r8a779m1-ulcb-CeX-9974.dts
| +- r8a779m1-ulcb-CeX-9987.dts
| +- r8a779m1-ulcb-CeX-9992.dts
| +- r8a779m1-ulcb-CeX-LVDS.dts
| +- ulcb-CeX.dtsi
+- drivers/media/
| +- i2c/
| | +- tw8842/      /* for raa278842 driver */
| | +- tw8846/      /* for raa278846 driver */
| | +- tw9992.c
| | +- isl79987.c
| | +- raa279972.c raa279972-reg.h raa279972-ReadMe.txt
| | +- raa279974.c raa279974-reg.h raa279974-ReadMe.txt
| +- platform/rcar-vin/
| | +- rcar-core.c
| | +- rcar-csi2.c
| | +- rcar-dma.c
| | +- rcar-v4l2.c
| +- v4l2-core/
| | +- v4l2-ioctl.c
| | +- v4l2-mc.c
| +- ..

```

V4L2 driver has many features, but normally you need only file_operations and v4l2_ioctl_ops.

struct file_operations

.owner	THIS_MODULE
.open	rvin_open
.mmap	vb2_fop_mmap
.release	rvin_release
.poll	vb2_fop_poll

struct v4l2_ioctl_ops

ioctl	description
vidioc_querycap	query capability
vidioc_g_fmt_vid_cap	g_fmt
vidioc_s_fmt_vid_cap	s_fmt
vidioc_g_dv_timings	for HD video
vidioc_s_dv_timings	
vidioc_g_std	for SD video
vidioc_s_std	
vidioc_querystd	
vidioc_reqbufs	request video frame buffer with number. If number is 0, it deallocate the allocated buffers. It uses a value set by vidioc_s_fmt command.
vidioc_querybufs	query buffer to use on mmap
vidioc_qbuf	move buffer from userspace to drivers read queue
vidioc_dqbuf	get captured frame from driver's done queue to userspace.
vidioc_expbuf	Generate dmabuf fd
vidioc_streamon	Start streaming
vidioc_streamoff	Stop streaming

Below is a step-by-step used routine on application.

	VIN	csi2	Tx sub-device
open()	.open	poweron	poweron
VIDIOC_QUERYCAP	vidioc_querycap		
VIDIOC_S_FMT	vidioc_s_fmt_vid_cap		
VIDIOC_REQBUFS	vidioc_reqbufs		
VIDIOC_QUERYBUF	vidioc_querybufs		
mmap()	.mmap		
VIDIOC_QBUF	vidioc_qbuf with 4~6 buffers		
VIDIOC_STREAMON	vidioc_streamon	calculate UI check LP11 STREAMON	STREAMON
epoll	.poll		
VIDIOC_DQBUF	vidioc_dqbuf		
	vidioc_qbuf		
	..repeat epoll, DQBUF, and QBUF		
VIDIOC_STREAMOFF	vidioc_streamoff	STREAMOFF	STREAMOFF
unmap()			
VIDIOC_REQBUFS	vidioc_reqbufs with 0 to release		
close()	.release	poweroff	poweroff

4 V4L2 output driver (IMR driver)

IMR is for wide lens distortion correction and located between VIN and DISPLAY routine. It is a V4L2 device and, it receives frame from VIN device.

4.1 Open and check capability

In the R-Car platform, video device nodes are categorized based on their functional role in the hardware pipeline:

/dev/video0 ~ /dev/video3	Dedicated to VIN (Video Input) , responsible for capturing raw data from external camera sensors.
/dev/video4 ~ /dev/video7	Reserved for the IMR (Image Renderer) , a hardware accelerator used for image rectification, de-warping, and transformations.
Functional Link	In this implementation, we utilize /dev/video4 , which is internally linked with the output of /dev/video0 (VIN).

Because the IMR operates as a **Memory-to-Memory (M2M)** device—receiving data, processing it, and then outputting the result—the application must verify dual capabilities when opening /dev/video4:

V4L2_CAP_VIDEO_OUTPUT: Required to feed the raw video frames (received from VIN) into the IMR engine.

V4L2_CAP_VIDEO_CAPTURE: Required to retrieve the processed/transformed video frames.

```
struct v4l2_capability cap;
u32 caps;
int imr_fd;
```

```

imr_fd = open("/dev/video4", O_RDWR | O_NONBLOCK);
ret = ioctl(vfd, VIDIOC_QUERYCAP, &cap);
caps = cap.device_caps;
assert(caps & V4L2_CAP_VIDEO_OUTPUT);
assert(caps & V4L2_CAP_VIDEO_CAPTURE);
assert(caps & V4L2_CAP_STREAMING);

```

VIN driver	(VIN) => VIDEO_CAPTURE
IMR driver	VIDEO_OUTPUT => (IMR) => VIDEO_CAPTURE

4.2 set format

Set format for V4L2_BUF_TYPE_VIDEO_OUTPUT and V4L2_BUF_TYPE_VIDEO_CAPTURE.

```

struct v4l2_format fmt = {0};

fmt.type                = V4L2_BUF_TYPE_VIDEO_OUTPUT;
fmt.fmt.pix.width       = 1280;
fmt.fmt.pix.height      = 720;
fmt.fmt.pix.field       = V4L2_FIELD_NONE;
fmt.fmt.pix.pixelformat = V4L2_PIX_FMT_UYVY;
fmt.fmt.pix.bytesperline = 720 * 2;
fmt.fmt.pix.sizeimage    = 1280 * 720 * 2;
ret=ioctl(vfd, VIDIOC_S_FMT, &fmt);

memset(&fmt, 0, sizeof(fmt));
fmt.type                = V4L2_BUF_TYPE_VIDEO_CAPTURE;
fmt.fmt.pix.width       = 1280; /*or 1920 */
fmt.fmt.pix.height      = 720; /* or 1080 */
fmt.fmt.pix.field       = V4L2_FIELD_NONE;
fmt.fmt.pix.pixelformat = V4L2_PIX_FMT_UYVY;
fmt.fmt.pix.bytesperline = 720 * 2; /* or 1080 * 2 */
fmt.fmt.pix.sizeimage    = 1280 * 720 * 2; /* or 1920 * 1080 * 2 */
ret=ioctl(vfd, VIDIOC_S_FMT, &fmt);

```

4.3 input_memory_type

frame memory for input will be comes from VIN that is using MMAP. So IMR input frame memory will use V4L2_MEMORY_USERPTR or V4L2_MEMORY_DMABUF for input(_OUTPUT). And it will use V4L2_MEMORY_MMAP for output(_CAPTURE)

triple buffers. input, process, output.

VIN out (VIDEO_CAPTURE)	IMR in (VIDEO_OUTPUT)	IMR out (VIDEO_CAPTURE)
V4L2_MEMORY_MMAP	V4L2_MEMORY_USERPTR V4L2_MEMORY_DMABUF	V4L2_MEMORY_MMAP(recommend) V4L2_MEMORY_USERPTR V4L2_MEMORY_DMABUF
5 or 6 frame buffers	3 (or 2) frame buffers	3(or 2) frame buffers

```

#define IMR_POOL_SIZE 3

struct v4l2_requestbuffers reqbuf = {0};

memset(&reqbuf, 0, sizeof(reqbuf));
reqbuf.type = V4L2_BUF_TYPE_VIDEO_OUTPUT;
reqbuf.memory = V4L2_MEMORY_USERPTR; /* imr->input_memory_type; */
reqbuf.count = IMR_POOL_SIZE;
ret=ioctl(dev->fd, VIDIOC_REQBUFS, &reqbuf);

memset(&reqbuf, 0, sizeof(reqbuf));
reqbuf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
reqbuf.memory = V4L2_MEMORY_MMAP; /* imr->output_memory_type; */
reqbuf.count = count;
ret=ioctl(dev->fd, VIDIOC_REQBUFS, &reqbuf);

for(int i=0; i < IMR_POOL_SIZE; i++) {
    /* input(VIDEO_OUTPUT) is USERPTR, we do not need mmap */
    /* output(VIDEO_CAPTURE) is MMAP, we need a query and mmap */
    struct v4l2_buffer buf;
    memset(&buf, 0, sizeof(buf));

    buf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
    buf.memory = V4L2_MEMORY_MMAP;
    buf.index = i; /* buf_idx */
    ret=ioctl(imr_fd, VIDIOC_QUERYBUF, &buf);

    imr_pool[i].image_size = buf.length;
    imr_pool[i].offset[0] = buf.m.offset;
    imr_pool[i].plane[0] = mmap(NULL, buf.length,
        PROT_READ | PROT_WRITE, MAP_SHARED,
        imr_fd,
        buf.m.offset);
}

```

```

int imr_qbuf(int imr_fd, int buf_idx)
{
    struct v4l2_buffer buf;

    /* prepare in buffer */
    buffer = buf_pool->input;
    meta = &buffer->meta;

    memset(&buf, 0, sizeof(buf));
    buf.index = buf_idx;
    buf.type = V4L2_BUF_TYPE_VIDEO_OUTPUT;
    if(imr->input_memory_type == V4L2_MEMORY_DMABUF) {
        buf.memory = V4L2_MEMORY_DMABUF;
        buf.m.fd = meta->dmafd[0]; /* mandatory */
    }
    else { /* assume as V4L2_MEMORY_USERPTR */
        assert(imr->input_memory_type == V4L2_MEMORY_USERPTR);
        buf.memory = V4L2_MEMORY_USERPTR;
    }
}

```



```

    buf.m.userptr = (unsigned long)meta->plane[0];
}
buf.length = meta->image_size;
buf.bytesused = meta->image_size; /* mandatory */
ret = ioctl(imr_fd, VIDIOC_QBUF, &buf);

/* prepare output buffer */
buffer = buf_pool->output;
meta = &buffer->meta;
memset(&buf, 0, sizeof(buf));
buf.index = buf_idx;
buf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;

if(imr->output_memory_type == V4L2_MEMORY_MMAP) {
    buf.memory = V4L2_MEMORY_MMAP;
} else if(imr->output_memory_type == V4L2_MEMORY_USERPTR) {
    buf.memory = V4L2_MEMORY_USERPTR;
    buf.m.userptr = (unsigned long)meta->plane[0];
}
else if(imr->output_memory_type == V4L2_MEMORY_DMABUF) {
    buf.memory = V4L2_MEMORY_DMABUF;
    buf.m.fd = meta->dmafd[0]; /* mandatory */
}
else {
    printf("ERROR!\n");
}
buf.length = meta->image_size;
ret = ioctl(imr_fd, VIDIOC_QBUF, &buf);
}

```

```

int imr_dqbuf(int imr_fd, int *buf_idx)
{
    /* dequeue input buffer */
    memset(&buf, 0, sizeof(buf));
    buf.type = V4L2_BUF_TYPE_VIDEO_OUTPUT;
    buf.memory = imr->input_memory_type;
    ret = ioctl(dev->fd, VIDIOC_DQBUF, &buf);

    *buf_idx = buf.index; /* save */
    if(buf.flags & V4L2_BUF_FLAG_ERROR)
        printf("Error imr in buffer%d\n", buf.index);

    /* dequeue output buffer */
    memset(&buf, 0, sizeof(buf));
    buf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
    buf.memory = imr->output_memory_type;
    ret = ioctl(dev->fd, VIDIOC_DQBUF, &buf);
    if(buf.flags & V4L2_BUF_FLAG_ERROR)
        printf("Error imr in buffer%d\n", buf.index);

    if(input_buf_idx != output_buf_idx) {
        printf("Error imr in buffer%d\n", buf.index);
    }
}

```

Let me explain simple logic for IMR main loop. It enables VIDIOC_STREAM_ON without frame buffer on IMR driver.

Note: For capturing applications it is customary to enqueue a number of empty buffers, to start capturing and enter the read loop. Here the application waits until a filled buffer can be dequeued, and re-enqueues the buffer when the data is no longer needed. Output applications fill and enqueue buffers, when enough buffers are stacked up output is started.

When we have an input frame from VIN, we execute imr_qbuf() to provide the frame buffer to IMR driver. And enable epoll routine.

Sometime later (maybe 2ms), epoll() has a DONE flag. We uses VIDIOC_DQBUF, and then move back input frame to VIN, and send result frame to DISPLAY.

```
int timeout = 40; /* 40ms */

ret = imr_stream_on();

while(1) {
    if(kbhit())
        break;

    /* If we have in buffer, send to IMR driver */
    if(vqueue_is_empty(&imr_in_queue) == 0) {
        buffer = vqueue_pop_front(&imr_in_queue);
        buf_pool = imr_pool[imr_pool_buf_idx];
        buf_pool->input = buffer;

        ret = imr_qbuf(imr_fd, imr_pool_buf_idx);
        /* update pool_buf_index. */
        imr_pool_buf_idx++;
        imr_pool_buf_idx = imr_pool_buf_idx % IMR_POOL_SIZE;

        if(dev->sumbitted++ == 0)
            ret = imr_register_poll(imr_fd, 1);
    }
    /* epoll to check DONE flag */
    r = epoll_wait(imr_efd, 1, timeout);
    if(r < 0)
        break; /* ERROR */
    if(r == 0)
        continue; /* timeout, retry */

    for(k=0; k < r; k++) {
        assert((int)event[k].data.u32 == 0); /* example uses 1 IMR driver */
        ret = imr_process_buffer(...);
    }
}

ret = imr_stream_off();
```

below is a imr_process_buffer() sample code.

At first, it executes imr_dqbuf() to get the buf_idx value, and then disable epoll.

The in (VIDEO_OUTPUT) buffer is comes from VIN, and need to send back frame to VIN. So, it executes the buffer->dispose() callback function.

```
int imr_process_buffer()
{
    int buf_idx;
    ret = imr_dqbuf(imr_fd, &buf_idx, NULL, NULL, NULL);

    if(--dev->submitted == 0)
        ret = imr_register_poll(imr_efd, 0);

    buffer = buf_poop->input;
    if(buffer->dispose)
        ret = buffer->dispose(buffer);

    /* send to DISPLAY consumer */
    buffer = buf_pool->output;
    ret = vqueue_push_rear(&display_queue, buffer);
}
```

In case of the out (VIDEO_CAPTURE) buffer, we do not need callback function. We will do QBUF only when we have an in (VIDIOC_OUTPUT) from VIN.

5 Media Controller and sub-devices

So far we learned about VIN, and IMR.

On the real environment, we have a camera driver and MIPI CSI2 driver. And we have to know some more about Media Controller (MC) logic.

If we are using the camera bridge chip, logical connection will be below diagram.

CAMERA => (MIPI_CSI)	=> MIPI_CSI RX => VIN driver	=====> GPU => DRM
cable		=> IMR =>

5.1 media device model.

Discovering a device internal topology, and configuring it at runtime, is one of the goals of the media controller API.

To achieve this, hardware devices and Linux Kernel interfaces are modeled as graph objects on an oriented graph.

The object types that constitute the graph are:

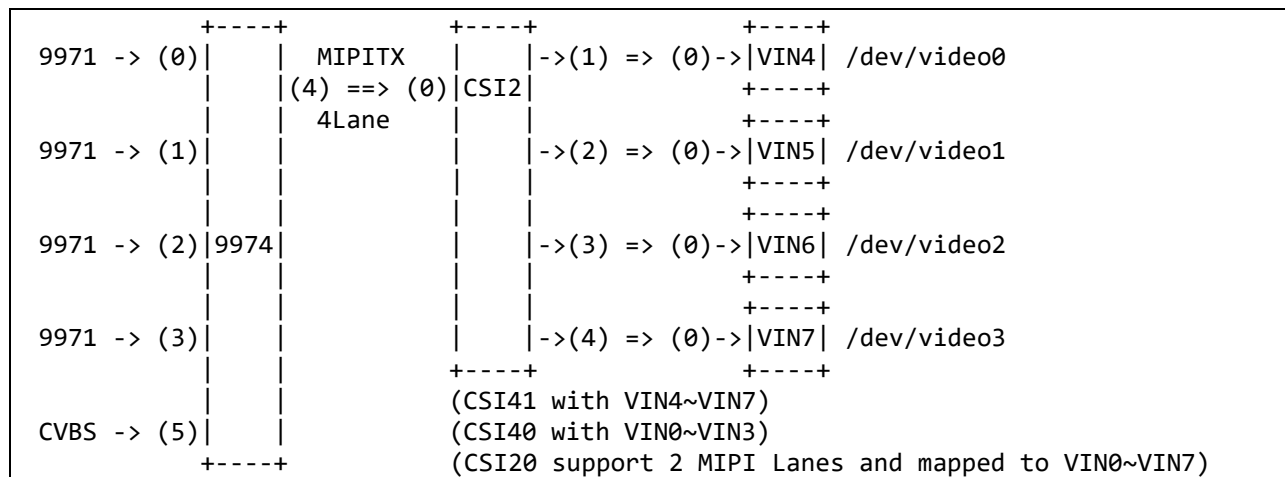
	Description
entity	a basic media hardware or software building block. It can correspond to a large variety of logical blocks such as physical hardware devices (CMOS sensor for instance), logical hardware devices (a building block in a System-on-Chip image processing pipeline), DMA channels or physical connectors.
interface	a graph representation of a Linux Kernel userspace API interface, like a device node or a sysfs file that controls one or more entities in the graph.

pad	a data connection endpoint through which an entity can interact with other entities. Data (not restricted to video) produced by an entity flows from the entity's output to one or more entity inputs. Pads should not be confused with physical pins at chip boundaries.
data link	a point-to-point oriented connection between two pads, either on the same entity or on different entities. Data flows from a source pad to a sink pad.
interface link	a point-to-point bidirectional control connection between a Linux Kernel interface and an entity.

5.2 Topology

For example, below diagram is the Media Topology for RAA279974 on R-CAR H3SK.

RAA279974 is a camera bridge chip that can support 4 Camera inputs through RAA279971, and send through MIPI 4 Lanes. MIPI CSI2 Rx receives signals and then uses the Virtual Channel ID to map the video channel to VIN driver.



At first, execute "media-ctl -p" command to see the current status. And then, reconnect CSI2 and VIN driver. The last step is, setup format value on CSI2.

```
# media-ctl -p
# media-ctl -r
# media-ctl -l "'rcar_csi2 feab0000.csi2':1-> 'VIN4 output':0 [1]"
# media-ctl -l "'rcar_csi2 feab0000.csi2':2-> 'VIN5 output':0 [1]"
# media-ctl -l "'rcar_csi2 feab0000.csi2':3-> 'VIN6 output':0 [1]"
# media-ctl -l "'rcar_csi2 feab0000.csi2':4-> 'VIN7 output':0 [1]"
# media-ctl -V "'rcar_csi2 feab0000.csi2':0 [fmt:UYVY8_2X8/1280x720 field:none]"
```

On the below example, "BLUE" color area is needed to update for the 720P30+4CH.

If you execute "media-ctl -V" command on sink, media-ctl executes "-V" command to the source pad within same entity. We do not need "-V" command on pad1, pad2, pad3, pad4 on the "rcar_csi2 feab0000.csi2" entity.

```
root@h3ulcb:~# media-ctl -p
Media controller API version 5.10.41

Media device information
-----
driver          rcar_vin
```

```
model          renesas,vin-r8a7795
serial
bus info       platform:e6ef4000.video
hw revision    0x0
driver version  5.10.41
```

Device topology

```
- entity 1: rcar_csi2 feab0000.csi2 (5 pads, 9 links)
    type V4L2 subdev subtype Unknown flags 0
    device node name /dev/v4l-subdev0
    pad0: Sink
        [fmt:UYVY8_2X8/1280x720 field:none]
        <- "raa279974 4-003d AHL":4 [ENABLED,IMMUTABLE]
    pad1: Source
        [fmt:UYVY8_2X8/1280x720 field:none]
        -> "VIN4 output":0 [ENABLED]
        -> "VIN5 output":0 []
        -> "VIN6 output":0 []
    pad2: Source
        [fmt:UYVY8_2X8/1280x720 field:none]
        -> "VIN4 output":0 []
        -> "VIN5 output":0 [ENABLED]
        -> "VIN7 output":0 []
    pad3: Source
        [fmt:UYVY8_2X8/1280x720 field:none]
        -> "VIN6 output":0 [ENABLED]
    pad4: Source
        [fmt:UYVY8_2X8/1280x720 field:none]
        -> "VIN7 output":0 [ENABLED]

- entity 7: raa279974 4-003d AHL (6 pads, 1 link)
    type V4L2 subdev subtype Unknown flags 0
    device node name /dev/v4l-subdev1
    pad0: Sink
        [fmt:UYVY8_2X8/1280x720@1/30 field:none colorspace:rec709]
        [dv.caps:BT.656/1120 min:720x480@13500000 max:...]
    pad1: Sink
        [fmt:UYVY8_2X8/1280x720@1/30 field:none colorspace:rec709]
        [dv.caps:BT.656/1120 min:720x480@13500000 max:...]
    pad2: Sink
        [fmt:UYVY8_2X8/1280x720@1/30 field:none colorspace:rec709]
        [dv.caps:BT.656/1120 min:720x480@13500000 max:...]
    pad3: Sink
        [fmt:UYVY8_2X8/1280x720@1/30 field:none colorspace:rec709]
        [dv.caps:BT.656/1120 min:720x480@13500000 max:...]
    pad4: Source
        [fmt:UYVY8_2X8/1280x720@1/30 field:none colorspace:rec709]
        [dv.caps:BT.656/1120 min:720x480@13500000 max:...]
        [dv.detect:BT.656/1120 1280x720p30 (1650x750) stds:...]
        [dv.current:BT.656/1120 1280x720p30 (1650x750) stds:...]
        -> "rcar_csi2 feab0000.csi2":0 [ENABLED,IMMUTABLE]
    pad5: Sink
        [fmt:UYVY8_2X8/720x420 field:interlaced-tb colorspace:smpte170m]
        [dv.caps:BT.656/1120 min:720x480@13500000 ... caps:interlaced]
```

```

- entity 20: VIN4 output (1 pad, 2 links)
    type Node subtype V4L flags 0
    device node name /dev/video0
    pad0: Sink
        <- "rcar_csi2 feab0000.csi2":1 [ENABLED]
        <- "rcar_csi2 feab0000.csi2":2 []

- entity 24: VIN5 output (1 pad, 2 links)
    type Node subtype V4L flags 0
    device node name /dev/video1
    pad0: Sink
        <- "rcar_csi2 feab0000.csi2":2 [ENABLED]
        <- "rcar_csi2 feab0000.csi2":1 []

- entity 28: VIN6 output (1 pad, 2 links)
    type Node subtype V4L flags 0
    device node name /dev/video2
    pad0: Sink
        <- "rcar_csi2 feab0000.csi2":1 []
        <- "rcar_csi2 feab0000.csi2":3 [ENABLED]

- entity 32: VIN7 output (1 pad, 2 links)
    type Node subtype V4L flags 0
    device node name /dev/video3
    pad0: Sink
        <- "rcar_csi2 feab0000.csi2":2 []
        <- "rcar_csi2 feab0000.csi2":4 [ENABLED]

root@h3ulcb:~#

```

	type	dev node name	
VIN	V4L dev	/dev/video0 ~ /dev/video3	video capture driver
CSI2	subdev	/dev/v4l-subdev0	MIPI CSI2 receiver
raa279974	subdev	/dev/v4l-subdev1	Camera bridge with MIPI CSI2 Transmitter
IMR	V4L dev	/dev/video4 ~ /dev/video7	Wide Lens distortion correction engine

5.3 tools

'media-ctl' is a primary tool to control the media controller.

options	Description
-p	Print current status
-r	Reset to default.
-l	Link between Source and Sink pads
-V	VIDIOC_S_FMT command
-d	select media, default is '/dev/media0'.
--help	Print help messages

'v4l2-ctl' is the other tool that is provided by media control. It provides more options for the debug purpose.

```

root@v3x:~# dmesg -c
root@v3x:~# v4l2-ctl -d /dev/v4l-subdev1 --get-subdev-fmt 0

```

```

ioctl: VIDIOC_SUBDEV_G_FMT (pad=0)
  Width/Height      : 1280/720
  Mediabus Code     : 0x2006 (MEDIA_BUS_FMT_UYVY8_2X8)
  Field             : None
  Colorspace        : Rec. 709
  Transfer Function : Default (maps to Rec. 709)
  YCbCr/HSV Encoding: Default (maps to Rec. 709)
  Quantization      : Default (maps to Limited Range)

```

```

root@v3x:~# dmesg -c
[ ] raa279974_ops_get_format Active pad:0
[ ] SIL9135 ch0 Input 1280x720 74*2MHz 8bit
[ ] 9971 ch0 1280x720@60 74MHz total:1650x750 on 8bit BUS
[ ] 9974 ch0 total:1649x750 on 8bit BUS
[ ] raa279974_ops_get_format 1280x720 UYVY_2X8 NONE:1 + input_pad:0 4Lane @100/6000

```

Use 'dmesg' command to see more debug messages.

5.4 Media Bus

Packed YUV format.

MEDIA_BUS_FMT_UYVY8_2X8	0x2006	2 Times, Use 8 lines. Use double clock
MEDIA_BUS_FMT_UYVY8_1X16	0x200f	Use 16 lines to send one pixel.

On the previous example, we are using “[fmt:UYVY8_2X8/1280x720 field:none]”. On V4L2, it request using same format between sender and receiver entities.

5.5 libmedia and application

On cam-thread application program, it search “raa279974” and change max device number as 4. And, it generates below set-up scripts and execute.

```

# media-ctl -r
# media-ctl -l "'rcar_csi2 feab0000.csi2':1-> 'VIN4 output':0 [1]"
# media-ctl -l "'rcar_csi2 feab0000.csi2':2-> 'VIN5 output':0 [1]"
# media-ctl -l "'rcar_csi2 feab0000.csi2':3-> 'VIN6 output':0 [1]"
# media-ctl -l "'rcar_csi2 feab0000.csi2':4-> 'VIN7 output':0 [1]"
# media-ctl -V "'rcar_csi2 feab0000.csi2':0 [fmt:UYVY8_2X8/1280x720 field:none]"

```

6 Camera bridge video drivers (V4L2 sub-devices)

I created device drivers for the below video bridge chip

	Input	output	Description
TW9992	CVBS	MIPI 2x, BT656	
ISL79987	4x CVBS	MIPI 4x BT656	
RAA278846	CVBS, DTV, LVDS	MIPI 4x BT656 LVDS	Multi-input, Multi-output bridge chip

RAA278842	CVBS, DTV, LVDS	MIPI 4x BT656 LVDS	Multi-input, Multi-output bridge chip
RAA279972	AHL, CVBS	MIPI 4x BT656	AHL(Analog HD Link) to replace SerDes.
RAA279974	4x AHL, CVBS	MIPI 4x BT656	4 channel AHL.

6.1 DTB and activate

When you create device driver, add the name on the “.compatible”, Linux will search that name on DTB(DTS), and then probe your driver.

device driver	DTS
<pre>static const struct of_device_id raa279974_of_table[] = { { .compatible = "raa,raa279974", }, { } }; MODULE_DEVICE_TABLE(of, raa279974_of_table);</pre>	<pre>&i2c4 { raa279974@30D { ... } };</pre>

6.2 Probe

probe()	Description
<pre>state->dev = &client->dev; state->client = client; i2c_set_clientdata(client, state); raa279974_check_chip_id(state);</pre>	Setup I2C client data. Read I2C device ID. If failed, enable the dummy mode!
<pre>raa279974_parse_dt(state); raa279974_init_chip_registers(state); raa279974_init_clients(state);</pre>	Read DTB information Download default register values Download local client register values with ControlChannel.
<pre>v4l2_subdev_init(&state->sd, &raa279974_ops); state->sd.flags = V4L2_SUBDEV_FL_HAS_DEVNODE; state->sd.owner = state->dev->driver->owner; state->sd.dev = state->dev; v4l2_set_subdevdata(&state->sd, state);</pre>	Init subdev, and link ops. Set subdev data.
<pre>snprintf(state->sd.name, sizeof(state->sd.name), "%s %d-%04x %s", state->dev->driver->name, i2c_adapter_id(state->client->adapter), state->client->addr, "AHL"/*identity*/);</pre>	Prepare unique sub-device name, for example “raa279974 4-003d AHL”. 4 means I2C4 I2C adapter. 003d means I2C ID with 7bit.
<pre>media_entity_pads_init(&state->sd.entity, RAA279974_CSI2_NR_PADS, state->pads); raa279974_init_controls(state);</pre>	initialize the media_entity struct for media framework. Set PIXEL_RATE, BRIGHTNESS, CONTRAST, SATURATION, HUE, SHARPNESS, and TEST Pattern controls
<pre>v4l2_async_register_subdev(&state->sd);</pre>	registers a sub-device to the asynchronous subdevice framework
<pre>raa279974_set_pixelrate(&state->sd, state->m_timings[RAA279974_PORT_CSI2].bt.pixelclock);</pre>	37.125MHz for 720P30.

6.3 Interface Operations

v4l2_subdev_ops	Description
core_ops	
.reset	Support SW Reset
.g_register .s_register	DEBUG feature to read/wtite register value
s_power	Power ON/OFF
video_ops	
s_routing	Select input() so output ()
g_routing	VSPAUTO extension to read-back s_routing value
g_std	Get video standard on SD
s_std	Set video standard on SD
querystd	find current video standard with measure on SD
g_tnorm	Enumerate SD modes
g_input_status	print input status.
s_stream	for STREAMON and STREAMOFF
g_frame_interval	Get Frame Per Sec (Freq)
s_frame_interval	Set Frame Per Sec
s_dv_timing	Set Digital timing mode
g_dv_timing	Get Digital timing mode
query_dv_timings	Find current Digital input mode.
g_mbus_config s_mbus_config	VSPAUTO extension for mbus (media bus). V4L2 starts using get_mbus_config.
pad_ops	
enum_mbus_code	Enumerate mbus (media bus)
enum_frame_size	Enumerate frames size
enum_frame_intervalus	Enumerate frame frequency
get_fmt	Get selected format values
set_fmt	Set format values
dv_timings_cap	Get DV timing capabilities.
enum_dv_timings	Enumerate supported digital timings
get_mbus_config	get mbus config. Added from 5.x.x
media_entity_operation	Description
link_validate	v4l2_subdev_link_validate().This function ensures that width, height and the media bus pixel code are equal on both source and sink of the link.

V4L2 devices can be opened more than once. Users can for example start a “panel” application to change controls like brightness, while another application captures video.

v4l2_ctrl_handlers	Description
s_ctrl	Set below controls
VIDIOC_CID_PIXELRATE	Set and provide the pixelrate to MIPI CSI2 receiver. Receiver reads PIXELRATE, number of data lane, and number of Virtual Channel to calculate the UI (Unit Interval) value.
VIDIOC_CID_BRIGHTNESS	For Brightness control
VIDIOC_CID_CONTRAST	For Contrast control
VIDIOC_CID_SATURATION	For Saturation control

VIDIOC_CID_HUE	For Hue control
VIDIOC_CID_SHARPNESS	For Sharpness control
VIDIOC_CID_TEST_PATTERN	Enable & Select Test Pattern
VIDIOC_CID_9971_CFG	VSPAUTO extension to control AHL Tx chip.

v4l2_subdev_internal_ops	Description
.registered	Callback to confirm the csi2 registered function status

6.4 remove

```
static int raa279974_remove(struct i2c_client *client)
{
    struct raa279974_state *state = i2c_get_clientdata(client);

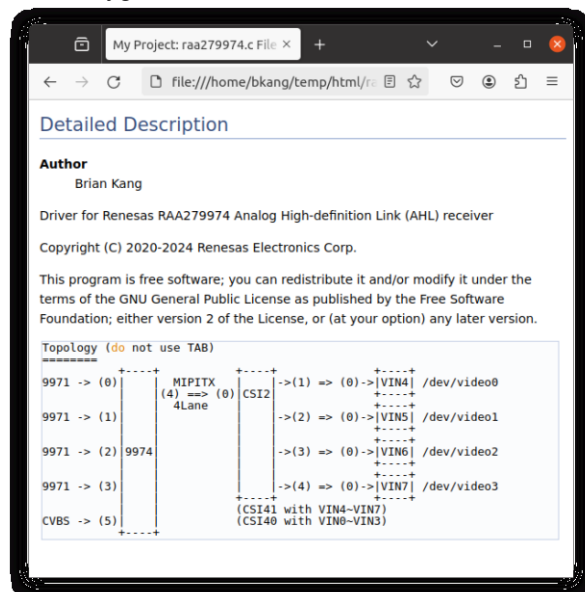
    v4l2_async_unregister_subdev(&state->sd);
    v4l2_ctrl_handler_free(&state->ctrl_hdl);
    media_entity_cleanup(&state->sd.entity);
    raa279974_unregister_clients(state);
    raa279974_dt_cleanup(state);
    mutex_destroy(&state->mutex);
    kfree(state);
    return 0;
}
```

6.5 Doxygen Ready

RAA279974.c has 9400 lines, and RAA9974-reg.h has 2200 lines.

RAA279974.c is on <https://github.com/vspauto/VideoDrivers/blob/main/raa279974.c>.

Use 'doxygen' to create HTML documentation.



It provides all command examples those you need to use 'media-ctl' and 'v4l2-ctl', even GStreamer commands.

7 DMABUF

We have 3 buffer types.

scattered in both the physical and (kernel) virtual address spaces	/* Physically scattered */ <media/videobuf-dma-sg.h> Normally requires hardware which can do scatter/gather DMA operations.
physically scattered, but which are virtually contiguous;	/* vmalloc() buffers */ <media/videobuf-vmalloc.h> Allocated with vmalloc(). These buffers are just as hard to use for DMA operations, but they can be useful in situations where DMA is not available but virtually-contiguous buffers are convenient.
physically contiguous	/* Physically contiguous */ <media/videobuf-dma-contig.h> Allocation of this kind of buffer can be unreliable on fragmented systems, but simpler DMA controllers cannot deal with anything else.

Use physically contiguous memory for DMA. That is why I prefer use MMAP on VIN when I use DMABUF.

Methods to use a physically contiguous memory

	Description
DMA-BUF Heap	Open /dev/dma_heap/linux, cma. Use DMA_HEAP_IOCTL_ALLOC to get a FD Map it to userspace using mmap()
V4L2/DRM Dumb buffers	Use DRM_IOCTL_MODE_CREATE_DUMB. Use V4L2 MMAP.
CMA Contiguous Memory Allocator	CMA is a kernel-side feature that reserves a large area of RAM at boot. It allows the kernel to use that memory for normal tasks (like page cache) until a driver specifically requests a "contiguous" block, at which point the kernel "evicts" the movable pages to satisfy the request.

Export DMABUF

```
#if USE_DMABUF
    for(i = 0; i < NUM_OF_BUFFERS; i++) {
        struct v4l2_exportbuffer expbuf = {0};
        expbuf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
        expbuf.index = i;
        ret = ioctl(vdev->fd, VIDIOC_EXPBUF, &expbuf);
        vin_pool[i].dmafd[0] = expbuf.fd;
    }
#endif
```

Example: Init DMABUF

```
struct v4l2_requestbuffers reqbuf;

memset(&reqbuf, 0, sizeof (reqbuf));
reqbuf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
reqbuf.memory = V4L2_MEMORY_DMABUF;
reqbuf.count = NUM_OF_BUFFERS;

ret = ioctl(fd, VIDIOC_REQBUFS, &reqbuf);
```

Example: Queue

```
int buffer_queue(int v4lfd, int index, int dmafd)
{
    struct v4l2_buffer buf;

    memset(&buf, 0, sizeof buf);
    buf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
    buf.memory = V4L2_MEMORY_DMABUF;
    buf.index = index;
    buf.m.fd = dmafd;

    if (ioctl(v4lfd, VIDIOC_QBUF, &buf) == -1) {
        perror("VIDIOC_QBUF");
        return -1;
    }

    return 0;
}
```

7.1 Refcount issue

When you do VIDIOC_EXPBUF, refcount becomes 1.

When you do VIDIOC_REQBUFS with "reqbuf.memory = V4L2_MEMORY_DMABUF, and reqbuf.count", refcount is increased.

When you do VIDIOC_REQBUFS with reqbuf.count 0, refcount is decreased.

When you close(dmafd) and refcount was 1, Kernel finally release memory.

If refcount was not 1, it decreases refcount only.

So I add some special routine on V4L2 and use "VIDIOC_EXPBUF+0x40" to force to release memory.

7.2 Fence on DMA

DMA is done with Hardware. If reader uses data before sender finishes the write, data tearing error will be happened. So solve, we can add OUT Fence on Sender, and use that on Receiver as IN Fence.

7.2.1 Add out-fence on Sender side.

Create fence_fd when it do VIDIOC_QBUF.

```
#include <linux/videodev2.h>
#include <sys/ioctl.h>
#include <unistd.h>
#include <stdio.h>

// to generate the fence fd that indicates the dma done state.
int qbuf_with_fence(struct vin_data *vin, int cam_idx, int buf_idx)
{
    struct vin_device *vdev = &vin->dev[cam_idx];
    VideoQueue_t *vqueue = &vdev->pool[buf_idx];
    struct v4l2_buffer buf = {0};

    buf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
    buf.memory = V4L2_MEMORY_DMABUF;
    buf.index = index;
    buf.flags = V4L2_BUF_FLAG_OUT_FENCE; /* to create fence */

    if (ioctl(vdev->fd, VIDIOC_QBUF, &buf) < 0) {
        perror("VIDIOC_QBUF failed");
        return -1;
    }

    // This file descriptor has a signal when DMA is done
    // From the recipient's perspective, this becomes 'In-fence'.
    vqueue->fence_fd = buf.request_fd;
    return 0;
}
```

7.2.2 Use fence on OpenGL Receiver.

On OpenGL, use fence_fd with EGL_SYNC_NATIVE_FENCE_FD_ANDROID, and close(fence_fd) when it is done.

```
#include <GLES2/gl2.h>
#include <GLES2/gl2ext.h>

/**
 * @param buffer_index: The index of the buffer to be rendered
 * (out of the 6-buffer static queue)
 * @param fence_fd: DMA completion fence FD received from V4L2 (VIN)
 */
void draw_camera_frame_with_fence(int buffer_index, int fence_fd) {
    EGLDisplay dpy = eglGetCurrentDisplay();

    // 1. In-fence Handling (Ensures GPU waits before rendering)
    if (fence_fd != -1) {
        // Create an EGL Sync object from the native fence_fd
        EGLint attribs[] = { EGL_SYNC_NATIVE_FENCE_FD_ANDROID, fence_fd, EGL_NONE };
        EGLSyncKHR sync = eglCreateSyncKHR(dpy, EGL_SYNC_NATIVE_FENCE_ANDROID, attribs);

        if (sync != EGL_NO_SYNC_KHR) {
            // Insert a command into the GPU pipeline to wait
            // until this fence is signaled (Non-blocking).
            // eglWaitSyncKHR does not block the CPU;
            // it only inserts a dependency into the GPU command queue.
            eglWaitSyncKHR(dpy, sync, 0);

            // The sync object can be destroyed immediately;
        }
    }
}
```

```

        // the GPU internal reference is maintained.
        eglDestroySyncKHR(dpy, sync);
    }
    // [Important] Ownership of the fd is transferred to eglCreateSyncKHR.
    // While driver implementations may vary,
    // the original fd is typically closed after creation.
    close(fence_fd);
}

// 2. Existing Rendering Logic (Zero-copy binding)
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_EXTERNAL_OES, camera_tex_ids[buffer_index]);

glViewport(0, 0, w, h);
glUseProgram(tex->program);

// Set Vertex Attribute
glEnableVertexAttribArray(0);
glEnableVertexAttribArray(1);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE,
    5 * sizeof(GLfloat), vVertices_vin_v_flip);
glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE,
    5 * sizeof(GLfloat), &vVertices_vin_v_flip[3]);

// 3. Execute Draw
glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_SHORT, indices);

// 4. (Optional) Create GPU Completion Fence (Out-fence)
// If needed, an Out-fence can be created
// via eglCreateSyncKHR(..., EGL_SYNC_NATIVE_FENCE_ANDROID, NULL)
// to notify the next stage (e.g., Display/DRM) that GPU rendering is complete.

//if you done every opengl rendering, execute eglSwapBuffers()
// eglSwapBuffers(dpy, eglGetCurrentSurface(EGL_DRAW));
}

```

7.2.3 on DRM

Add `fence_id` on `prop.in_fence_prop` before it executes the `drmModeAtomicCommit()`, and `close(fence_fd)` when it is done.

```

int drm_update_atomic()
{
    drmEventContext evctx = {
        .version = DRM_EVENT_CONTEXT_VERSION,
        .page_flip_handler = page_flip_handler,
    };
    int fence_fd = 0;

    next_bo = gbm_surface_lock_front_buffer(drm->gbm_surf);
    fb = drm_fb_get_from_bo(drm, next_bo);

    drmModeAtomicReqPtr req = drmModeAtomicAlloc();

    drmModeAtomicAddProperty(req, plane0_id, props.fb_id_prop, fb->fb_id);
    ..
    drmModeAtomicAddProperty(req, plane0_id, props.src_h_prop, 1080 << 16);
}

```

```

if(vqueue_empty() != true) {
    vqueue = vqueue_pop();
    /* update camera video frame on plane1 */

    drmModeAtomicAddProperty(req, plane1_id, props.fb_id_prop, fb->fb_id);
    ..
    drmModeAtomicAddProperty(req, plane1_id, props.src_w_prop, 1280 << 16);
    drmModeAtomicAddProperty(req, plane1_id, props.src_h_prop, 720 << 16);

    fence_fd = vqueue->fence_fd;
    if(fence_fd)
        drmModeAtomicAddProperty(req, plane_id1,
            props.in_fence_prop, (uint64_t)fence_fd);
}

uint32_t flags = DRM_MODE_ATOMIC_NONBLOCK | DRM_MODE_PAGE_FLIP_EVENT;
ret = drmModeAtomicCommit(drm->fd, req, flags, &waiting_for_flip);
drmModeAtomicFree(req);
if(fence_fd)
    close(fence_fd);

//wait event
while (waiting_for_flip) {
    // reuse fds that was assigned on drm_init_select().
    ret = select(drm->fd + 1, &drm->fds, NULL, NULL, NULL);
    if (ret <= 0) break;
    drmHandleEvent(drm->fd, &evctx);
}

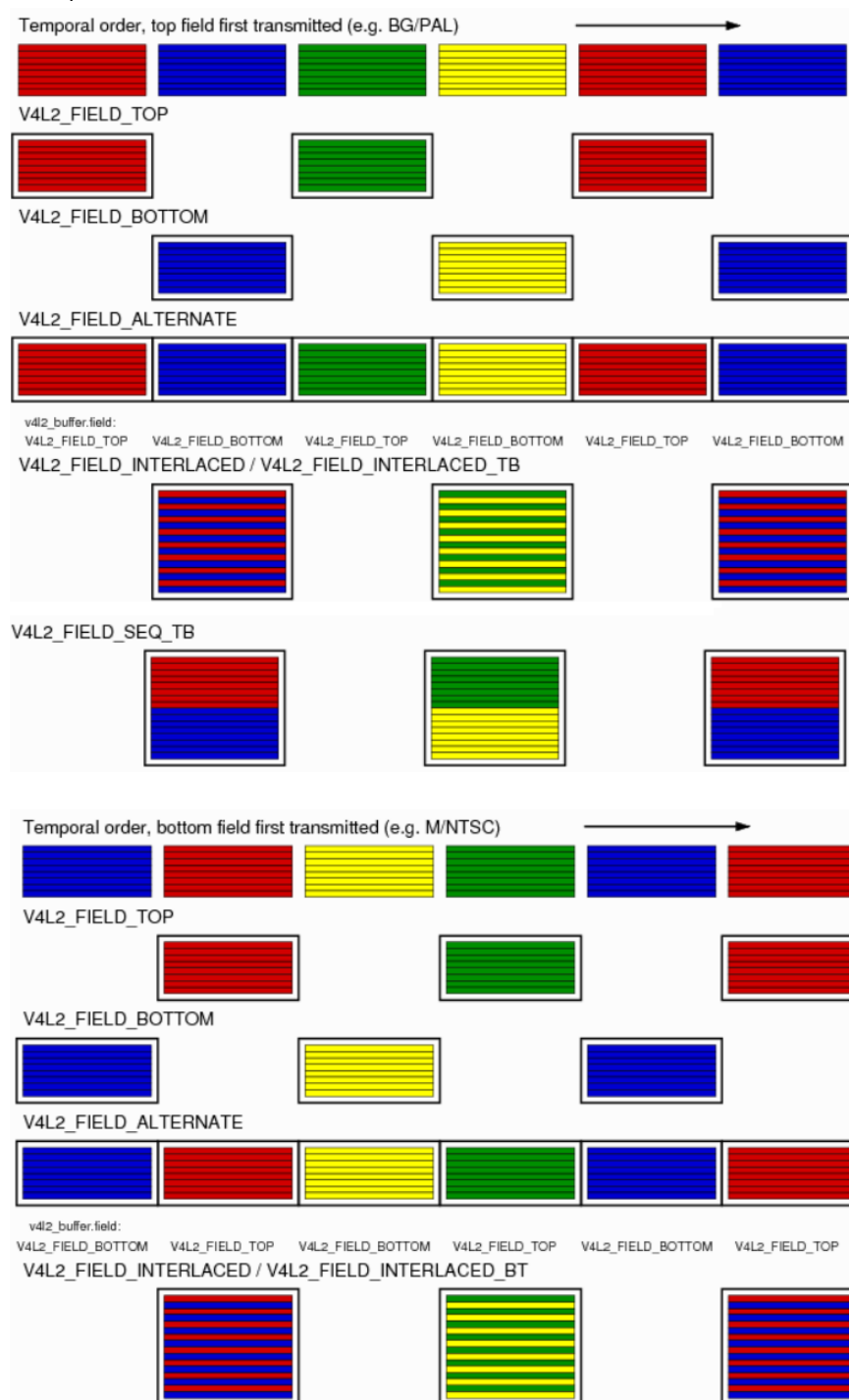
//replace buffer
gbm_surface_release_buffer(drm->gbm_surf, drm->bo);
drm->bo = next_bo;

return 0;
}

```

8 Interlaced Video (NTSC, PAL)

CVBS camera inputs are no longer in use. However, it is still important to be familiar with the following Field Order concepts.



Our private video IP always send Top field first, even on NTSC.